More on Foreign Keys

Lecture Notes · Game DB Junior

In this lesson

- Which table gets the **foreign key** column?
- Example: **comments** with user_id and photo_id
- **Primary keys vs foreign keys**: a comparison

1. Foreign keys for all relationships

We use the idea of a foreign key to set up **all four kinds** of relationships. One-to-many, many-to-one, many-to-many or one-to-one: we always use a **foreign key column**. That is how we set up relationships.

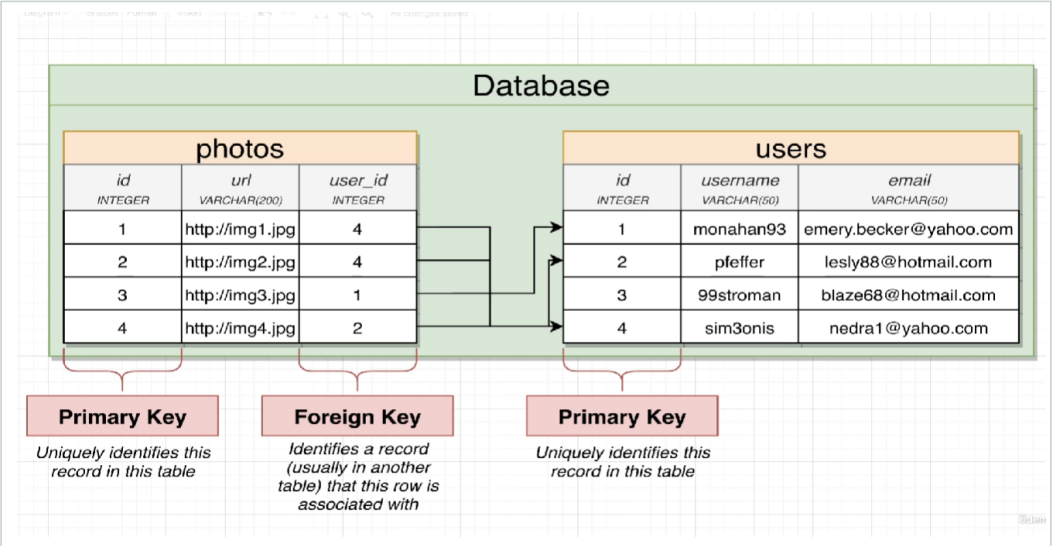


Figure 1 — Reminder: photos.user_id points at users.id

2. Which table gets the foreign key?

Let's look at how **comments** are related to **users** and to **photos**.

- A comment has **one** user, and a comment has **one** photo.
- From the other side: a user has **many** comments, and a photo has **many** comments.

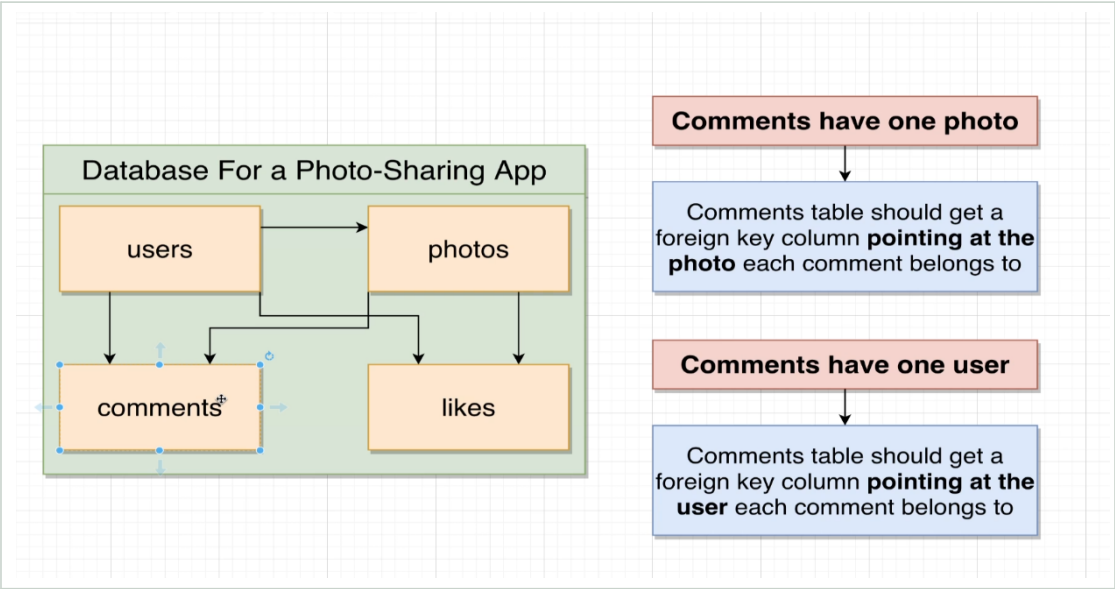


Figure 2 — Comments have one photo and one user

Rule: the **'many' side** of the relationship always gets the foreign key column.

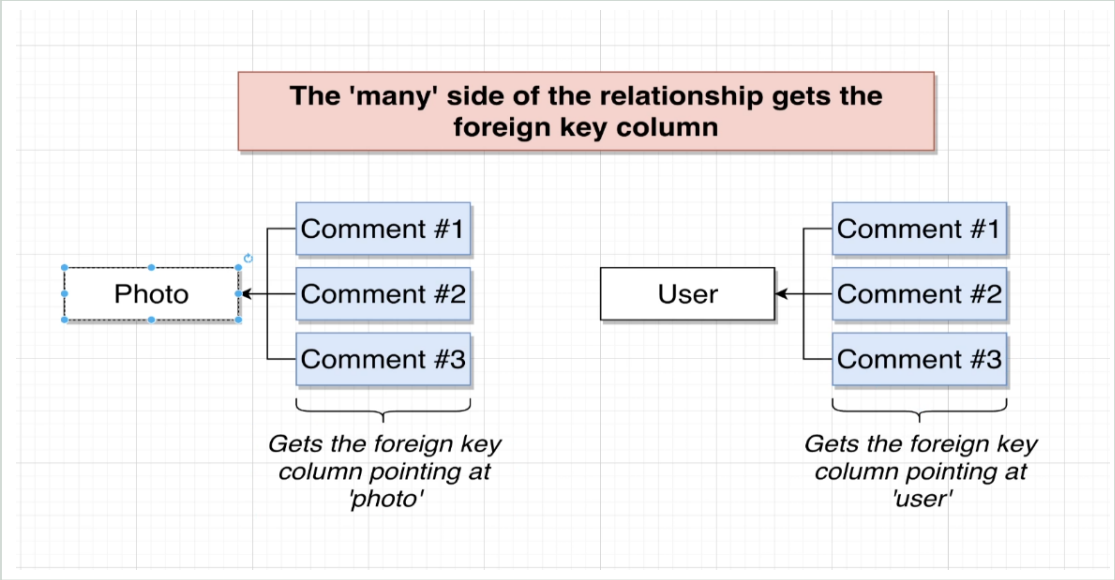


Figure 3 — The 'many' side gets the foreign key column

- Photos and comments: one-to-many. Comments are the many side. So the comments table gets a foreign key column **pointing at the photo** each comment belongs to.
- Users and comments: a user has many comments. Comments are the many side. So the comments table gets a foreign key column **pointing at the user** who created the comment.

3. The comments table

Now comments has **two foreign key columns**:

- **user_id** — the id of the user who created the comment.
- **photo_id** — the id of the photo the comment was added to.

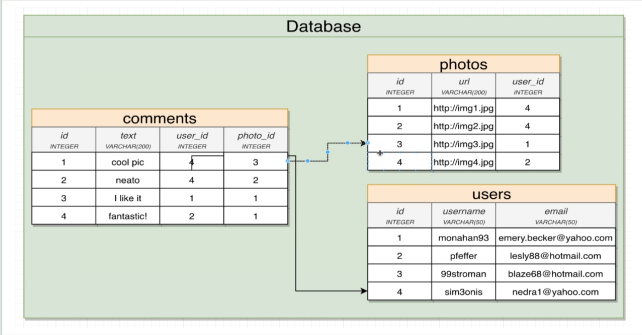


Figure 4 — Comment 1: user 4, photo 3

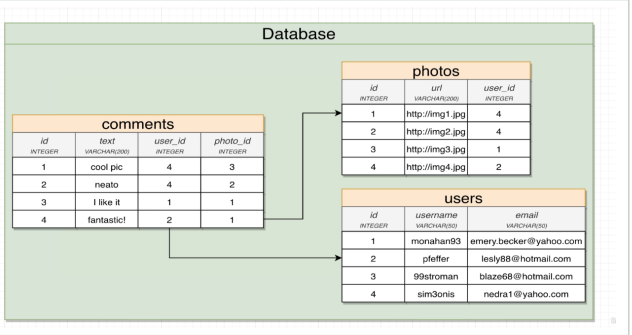


Figure 5 — Comment 4: user 2, photo 1

Comment	user_id → user	photo_id → photo
1 — cool pic	4 → sim3onis	3
2 — neato	4 → sim3onis	2
4 — fantastic!	2 → pfeffer	1

Is this really how it is done? Yes. This is the **only way** we set up relationships. It looks confusing at first, but it is a very **mechanical process**. It is always done the same way, so you only need to learn it correctly once.

4. Primary keys vs foreign keys

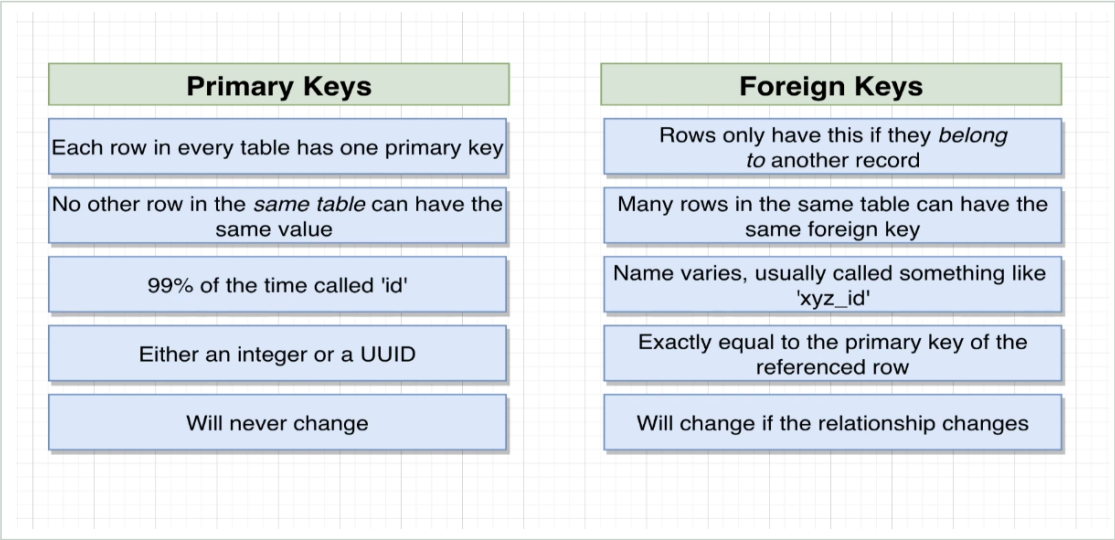


Figure 6 — Primary keys vs foreign keys

Notes on primary keys

- A primary key identifies one row. It is 100% unique: never a duplicate value in the column.

- Usually call it simply **id**. Some tutorials use a column like a country or city name as the primary key. This is a **bad idea**: many cities can have the same name, and a primary key must be unique.
- It is almost always an **integer** or a **UUID** (universally unique ID). A UUID looks like a random jumble of characters.
- It **never changes**. If we create a user with id 1, that id is never reused or given to another user.

Notes on foreign keys

- A table only needs a foreign key if its records **belong to** another record. In our schema, users do not belong to anything, so the users table will probably never have a foreign key. Almost everything else will.
- Foreign keys do **not** need to be unique. Example: two comments have user_id 4. That is fine.
- Naming can be different at each company. In this course: the name of the resource we point at + **_id**, like **user_id** and **photo_id**.
- The value is always **exactly equal** to the primary key of the row it points at.
- It **can change** if the relationship changes. Example: we could change the last comment's user_id from 2 to 1. Now that comment was created by a different user. This is allowed. Only the primary key must never change.

5. Summary

Idea	Remember
All relationships	Always set up with a foreign key column
Where it goes	The 'many' side of the relationship gets the foreign key
comments	Has user_id (who wrote it) and photo_id (which photo it is on)
Primary key	Unique, called id, integer or UUID, never changes
Foreign key	Can repeat, named xyz_id, equals the referenced primary key, can change

Next: a quick quiz, then we write real SQL to set up these relationships and use them to fetch data.

Sources: Lecture transcript and lecture images (Figures 1–6)